

Pizza Manager

- Billing & Item Management System

Database Structure

Handle invoices with multiple items

Many-to-Many Relationship:

One invoice can have many items. Each item belongs to one category.

Items Table

Items

- id (int, PK)
- name (string)
- category (string, e.g., Pizza, Topping, Beverage)
- price (float)

- In GORM, when embed gorm.Model in a struct, it provides the following fields:

ID (Primary Key) CreatedAt (Timestamp) UpdatedAt (Timestamp) DeletedAt (Timestamp, nullable)

Invoices Table

- one invoice per transaction

Invoices

- id (int, PK)
- customer_name (string)
- tax (float)
- total (float)

Invoice_Items Table

- stores item details for each invoice).

InvoiceItems

- id (int, PK)
- invoice_id (int, FK)
- item_id (int, FK)
- quantity (int)
- subtotal (float)

```
main.go handlers.go database.go models.go X
models.go > InvoiceItem
1 package main
2
3 import "gorm.io/gorm"
4
5 // items model
6 type Item struct {
7     gorm.Model
8     Name string `json:"name"`
9     Category string `json:"category"` // Pizza, Topping, Beverage
10    Price float64 `json:"price"`
11 }
12
13 // invoices model
14 type Invoice struct {
15     gorm.Model
16     CustomerName string `json:"customerName"`
17     Items []InvoiceItem `gorm:"foreignKey:InvoiceID" json:"items"`
18     Tax float64 `json:"tax"`
19     Total float64 `json:"total"`
20 }
21
22 // since 1-many
23 type InvoiceItem struct {
24     gorm.Model
25     InvoiceID uint `json:"invoiceId"`
26     ItemID uint `json:"itemId"`
27     Quantity int `json:"quantity"`
28     Subtotal float64 `json:"subtotal"`
29 }
30
```

- Database tables

```
main.go handlers.go database.go docker-compose.yml X
docker-compose.yml
1 version: "3.9"
2 services:
3     mysql:
4         image: mysql:latest
5         container_name: mysql-container
6         restart: always
7         environment:
8             MYSQL_ROOT_PASSWORD: MySQLRoot
9             MYSQL_DATABASE: goapi
10        ports:
11            - "3306:3306"
12        volumes:
13            - mysql_data:/var/lib/mysql
14
15 volumes:
16     mysql_data:
17
```

-
- Use MySQL as the database, run the instance through docker

```
mysql> USE goapi;
Database changed
mysql> SHOW TABLES;
Empty set (0.01 sec)

mysql> SHOW TABLES;
+-----+
| Tables_in_goapi |
+-----+
| invoice_items   |
| invoices        |
| items           |
+-----+
3 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM items;
+-----+-----+-----+-----+-----+-----+-----+
| id | created_at          | updated_at          | deleted_at          | name       | category | price |
+-----+-----+-----+-----+-----+-----+-----+
| 1 | 2025-01-07 13:03:58.617 | 2025-01-07 13:03:58.617 | 2025-01-07 22:03:12.782 | Chicken Pizza | Pizza    | 600 |
| 2 | 2025-01-07 13:04:59.452 | 2025-01-07 13:04:59.452 | NULL                | Coca-Cola    | Beverage | 300 |
| 3 | 2025-01-07 13:05:21.862 | 2025-01-07 13:05:21.862 | 2025-01-07 13:05:34.590 | Cheese pizza | Pizza    | 3500 |
| 4 | 2025-01-07 13:05:53.687 | 2025-01-07 21:30:34.555 | NULL                | Cheese pizza | Pizza    | 3560 |
| 5 | 2025-01-07 13:06:36.853 | 2025-01-07 21:33:16.038 | NULL                | Cheese       | Topping  | 2000 |
| 6 | 2025-01-07 21:57:29.559 | 2025-01-07 22:04:06.903 | NULL                | Pepperoni    | Topping  | 1500 |
+-----+-----+-----+-----+-----+-----+-----+
6 rows in set (0.01 sec)
```

- Took database content by accessing MySQL CLI in Docker

```
main.go handlers.go database.go X docker-compose.yml
database.go > ...
1 package main
2
3 import (
4     "fmt"
5     "gorm.io/driver/mysql"
6     "gorm.io/gorm"
7 )
8
9 var DB *gorm.DB
10 var err error
11
12 const DNS = "root:mysqlRoot@tcp(localhost:3306)/goapi?charset=utf8mb4&parseTime=True&loc=Local"
13
14 func InitialMigration() {
15     DB, err = gorm.Open(mysql.Open(DNS), &gorm.Config{})
16     if err != nil {
17         fmt.Println(err.Error())
18         panic("Cannot connect to DB")
19     }
20     DB.AutoMigrate(&Item{}, &Invoice{}, &InvoiceItem{})
21 }
22
```

- DB connection